

Toulouse INP - ENSEEIHT
École Nationale Supérieure d'Électrotechnique, d'Électronique, d'Informatique,
d'Hydraulique et des Télécommunications

RAPPORT DE STAGE DE FIN D'ÉTUDES
effectué chez Actimage

16 mars 2026 - 18 septembre 2026, 6 mois

Ingénierie Logicielle Full-Stack et DevOps

Vianney HERVY - Ingénieur Logiciel

vianney.hervy@etu.toulouse-inp.fr

Toulouse INP - ENSEEIHT
2 rue Charles Camichel
31071 Toulouse

Actimage
5 avenue Franco-Russe
75007 Paris

Responsable académique
Marc PANTEL
Maître de conférence en Informatique à l'IRIT /
ENSEEIHT

Encadrant chez Actimage
David KOLIN
(ex-)Directeur de l'agence Paris-Arcueil

Remerciements

Avant tout, je tiens à remercier mes deux encadrants sur cette expérience enrichissante qu'a été ce stage de fin d'étude. David pour son accompagnement, ses conseils et son humour, Matthias pour sa patience, sa bienveillance et ses succulents repas de fin de semaine.

Mes remerciements s'étendent à l'ensemble de mes collègues de l'agence Actimage Paris-Arcueil et d'ailleurs. Par ordre alphabétique : Aurélie, ma co-stagiaire solidaire pour son amitié, sa politesse et sa conversation ; Marine pour sa jovialité, sa créativité et son attention aux détails ; Romain pour sa curiosité et nos discussions technophiles ; et enfin Thomas, pour sa spontanéité et sa précieuse expertise footballistique.

Mes remerciements vont bien sûr également à Madeleine, ma fiancée, qui y est sûrement pour quelque chose dans la réussite de ce stage.

Sommaire

I	Introduction	5
I.1	Présentation de l'entreprise	5
I.1.1	Membres de l'entreprise	6
I.1.2	Équipe détaillée	6
I.1.3	Réalisations	7
I.1.4	Pôle développement	8
I.1.5	Pôle R et D	8
I.2	Contexte du stage	8
I.3	Gestion de projet	8
I.4	Poste de travail et outillage de développement	8
I.4.1	Environnement d'exécution et philosophie de travail	8
I.4.2	Outils collaboratifs et centralisation de la connaissance	8
I.4.3	Inspection web et débogage dynamique	9
I.4.4	L'éditeur de code: le choix de Neovim	10
I.4.5	Contributions aux logiciels libres et outillage sur mesure	10
I.4.6	Le terminal comme espace de travail unifié	11
I.4.7	Conteneurisation et exécution locale	11
I.4.8	Évolution du poste de travail	11
II	Projet Office national des combattants et des victimes de guerre ¹ (ONaCVG)	12
II.1	Introduction	12
II.1.1	Présentation du client et genèse du besoin	12
II.1.2	L'existant : un défi de taille et de structure de la donnée	12
II.1.3	Migration et regroupement familial	12
II.1.4	Refonte logicielle et application de gestion état civil militaire (ECM).	13
II.1.5	Contraintes d'interopérabilité	13
II.2	Migration et regroupement familial : le défi de la réconciliation des données	14
II.2.1	Choix technologiques et rigueur logicielle	14
II.2.2	Architecture transactionnelle et asynchrone	14
II.2.3	Stratégies de résolution des conflits	15
II.2.4	Ingénierie du déploiement et conteneurisation	15
II.2.5	Limites du modèle de données plat	16
II.3	Application principale : conception et réalisation	16
II.3.1	Architecture technique et paradigme « Front-End »	16
II.3.2	Rigueur logicielle et Intégration Continue (CI/CD)	16
II.3.3	Implémentation des règles métiers et sécurité	17
II.3.4	Défis techniques des modules fonctionnels	17
III	PIAWEB : Une histoire de DevOps	18
III.1	TODO: find title	18
III.1.1	Architecture de l'environnement DevOps et de l'infrastructure	18
III.1.2	Intégration Continue et Déploiement Continu (CI/CD)	19
III.1.3	Gestion des environnements et stratégie de branche	19
III.1.4	Le processus de livraison et de montée de version	20
III.2	TODO: find title	21
III.2.1	Le bogue	21
III.2.2	Appareillage sur lest	21
IV	Conclusion	22
IV.1	À propos de PHP/Symfony TODO: définir un titre	22

¹<https://www.onac-vg.fr>

IV.2 Enrichissement personnel	22
Bibliographie	23
Références logicielles	24
V Annexes	25

I Introduction

I.1 Présentation de l'entreprise

Actimage est une entreprise de services numériques (ESN) française créée en 1995 par Christophe Megel, l'actuel PDG. Le travail de l'entreprise est divisé en 3 pôles principaux communicants: le développement, le conseil et la recherche et développement (R et D). Mon stage s'est déroulé dans le pôle développement mais j'ai eu l'occasion d'interagir avec le pôle R et D sur certains sujets. Les effectifs d'Actimage sont répartis entre 8 agences dans 5 pays. Celles avec lesquelles j'ai le plus été en contact sont Paris, Arcueil, Colmar, Strasbourg, Metz et Luxembourg.

I.1.1 Membres de l'entreprise

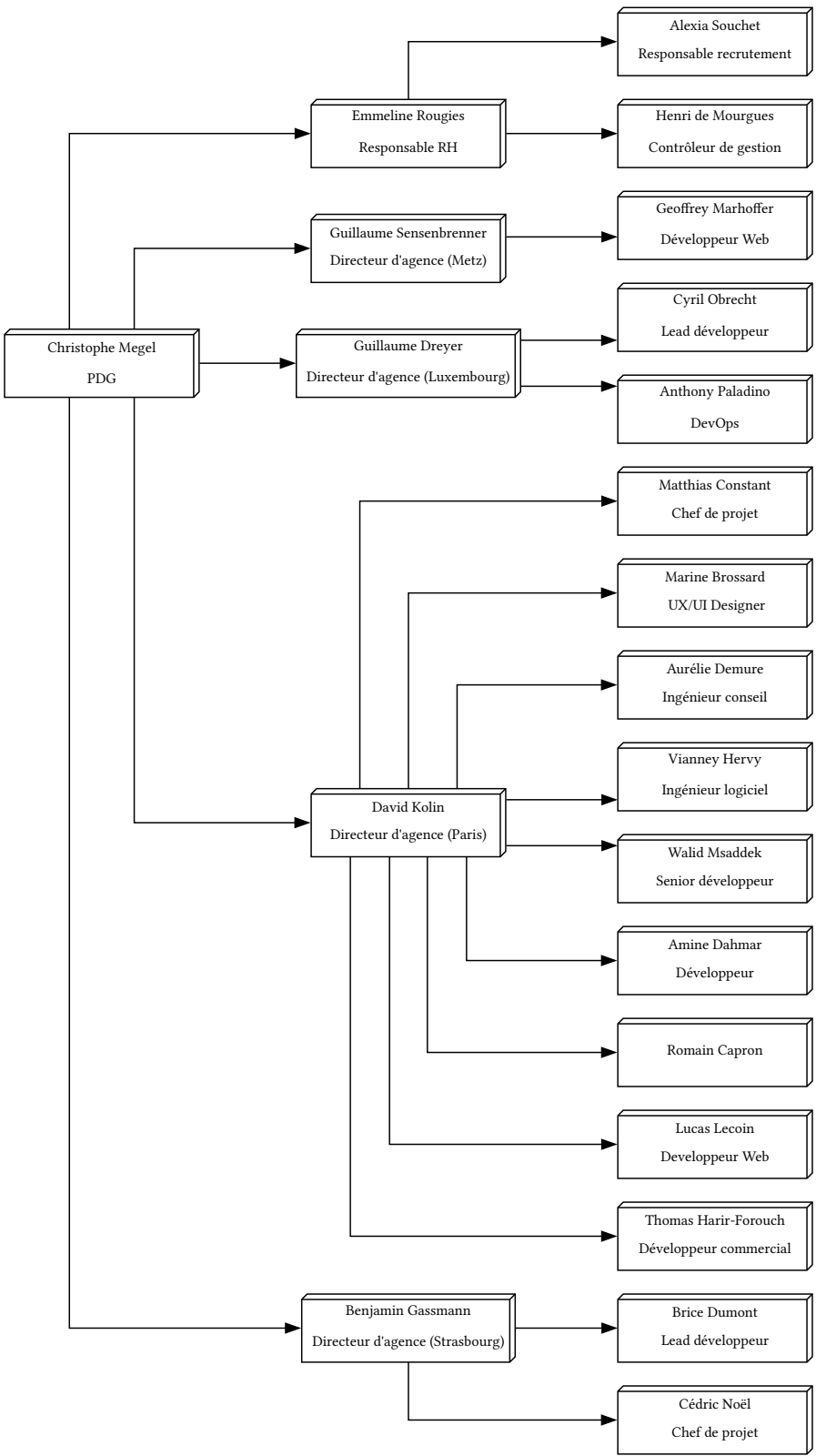


Fig. 1. – Organigramme de l'entreprise

I.1.2 Équipe détaillée

David Kolin (DKO)

TODO: présenter

Matthias Constant (MCO)

TODO: présenter

Marine Brossard (MBR)

TODO: présenter

Aurélie Demure (ADE)

TODO: présenter

Romain Capron (RCO)

TODO: présenter

Thomas Harir-Forouch

TODO: présenter

I.1.3 Réalisations

DSFR

L'expertise d'Actimage en développement brille tout particulièrement sur les marchés publics. L'entreprise a notamment participé à la conception du Système de Design de l'État Français² (DSFR). Le DSFR regroupe un ensemble de règles et de composants réutilisables pour les interfaces officielles des sites en `.gouv.fr`. Il permet à l'État d'offrir des services numériques simples, accessibles et reconnaissables. C'est notamment celui que vous retrouvez sur `impots.gouv.fr`, `ants.gouv.fr` et `sante.gouv.fr`.

Ce système de design est conçu pour être agnostique et modulaire. Il se décline sous plusieurs formats afin de couvrir tout le cycle de vie d'un projet de la phase de conception, au prototypage, à l'implémentation technique. Il existe notamment une librairie Figma, un socle de base en HTML/CSS/JS natif à travers le paquet `@gouvfr/dsfr` ainsi que des portages développés par la communauté tels que `@codegouvfr/react-dsfr` (React), `@gouvminint/vue-dsfr` (Vue), `ngx-dsfr` (Angular), `django-dsfr` (Django) et `drupal/ui_suite_dsfr` (Drupal).

Site officiel du Gouvernement

L'un des sites majeurs du gouvernement est `info.gouv.fr`. C'est aussi un des projets principaux d'Actimage qui en réalise le développement côté serveur et une partie du développement côté client. Plusieurs développeurs travaillent à temps plein sur ce projet, dont même certains physiquement au Service d'information du Gouvernement³ (SIG).

BDnf

Actimage est également l'entreprise derrière BDnF⁴, un outil ludo-éducatif de création de bandes dessinées et de récits multimédias développé pour le compte de la Bibliothèque nationale de France⁵ (BnF). Destinée principalement au milieu scolaire, l'application se connecte à la bibliothèque numérique Gallica via un module d'import permettant aux utilisateurs d'intégrer directement des corpus d'images d'archives dans leurs projets.

Techniquement, l'application a été développée de manière multi-support avec le moteur Unity 3D. L'architecture logicielle repose sur un noyau commun partagé entre les environnements de bureau (macOS, Windows) et tablettes (Android, iOS), tout en implémentant des fonctionnalités spécifiques adaptées aux

²<https://www.systeme-de-design.gouv.fr>

³<https://www.info.gouv.fr/organisation/service-d-information-du-gouvernement-sig>

⁴<https://bdnf.bnf.fr>

⁵<https://www.bnf.fr>

contraintes des versions mobiles. Afin d'accélérer le développement des nouvelles fonctionnalités et de garantir la cohérence des interfaces sur toutes ces plateformes, l'équipe s'est appuyée sur la mise en place d'un système de design strict. Enfin, la conception a fait l'objet de tests d'utilisabilité itératifs menés directement auprès d'élèves pour valider l'ergonomie.

Hol'Autisme

Actimage s'inscrit fortement dans l'innovation et la R et D avec des projets à fort impact sociétal à l'image de Hol'Autisme⁶. Ce projet novateur propose le premier catalogue d'applications en réalité mixte destiné à aider les enfants et adolescents atteints de troubles du spectre autistique à développer leurs compétences sociales. Développée notamment avec le moteur Unity pour le casque HoloLens, la solution permet de simuler des situations du public ou du quotidien dans un environnement interactif et contrôlé. Le but est d'aider les patients à appréhender les codes sociaux et à gagner progressivement en autonomie sans subir l'angoisse du monde réel.

L'expertise technologique du projet va bien au-delà de la simple réalité mixte : le système intègre un bracelet connecté permettant de mesurer le niveau d'anxiété de l'apprenant en temps réel, couplé à une plateforme web de contrôle et de suivi. Grâce à l'analyse de données et à des outils statistiques avancés, le personnel médico-éducatif peut analyser finement les sessions. La pertinence de ce dispositif global, dont la première preuve de concept s'intitule PopBalloons, a d'ailleurs été saluée par l'écosystème technologique, le projet étant lauréat des concours French IOT 2017 et Futur.e.s 2018.

I.1.4 Pôle développement

I.1.5 Pôle R et D

I.2 Contexte du stage

I.3 Gestion de projet

I.4 Poste de travail et outillage de développement

I.4.1 Environnement d'exécution et philosophie de travail

Au sein d'Actimage, le matériel mis à ma disposition était un poste opérant sous Windows 11. Afin de retrouver un environnement de développement familier, performant et conforme à la philosophie Unix qui guide mon flux de travail quotidien, j'ai fait le choix d'exploiter le sous-système Windows pour Linux [2]. J'y ai déployé une distribution Arch Linux. Cette approche m'a permis de répliquer quasi à l'identique l'environnement modulaire que j'utilise à titre personnel, tout en m'affranchissant des limitations inhérentes au système d'exploitation hôte pour le développement d'applications web.

I.4.2 Outils collaboratifs et centralisation de la connaissance

L'environnement de travail d'un ingénieur logiciel ne se limite pas à son éditeur de code et à son terminal. La dimension collaborative et la gestion de la connaissance sont des composantes tout aussi vitales, particulièrement au sein d'une ESN où de multiples acteurs (clients, chefs de projet, designers, développeurs) interagissent quotidiennement. Pour orchestrer cette communication, Actimage s'appuie principalement sur la suite Microsoft.

Le flux de communication synchrone était assuré par Microsoft Teams. Bien que cet outil ne bénéficiât d'aucune intégration automatisée avec notre chaîne d'intégration continue (comme des alertes webhooks liées à Jenkins ou GitLab), il constituait le centre névralgique de la vie d'équipe. C'est sur cette plateforme que se tenaient nos réunions quotidiennes de synchronisation (*daily stand-ups*). L'application était structurée en différents canaux dédiés aux projets, servant d'espaces d'échanges informels pour partager des extraits de

⁶<https://www.holautisme.com>

code, soumettre des idées d'architecture, formuler des remarques ou solliciter de l'aide auprès de développeurs plus expérimentés comme Brice ou Matthias.

Pour la communication asynchrone et formelle, Microsoft Outlook était de rigueur. Ce canal était privilégié pour les échanges directs avec les clients, souvent organisés via des listes de diffusion. La boîte de réception agissait également comme un agrégateur d'événements : j'y recevais les rappels de réunions, les annonces internes de l'entreprise, mais surtout les notifications automatisées générées par les outils de billetterie (*ticketing*) des clients. Ces alertes me permettaient de suivre en temps réel la progression des tickets d'anomalie ou l'évolution d'un fil de discussion lié à une spécification fonctionnelle.

Concernant la gestion documentaire, une dichotomie marquée existait entre les aspects « métiers » et les aspects purement « techniques » des projets. Toute la documentation fonctionnelle et contractuelle était centralisée sur SharePoint. On y retrouvait une arborescence dense composée de documents Microsoft Word pour les spécifications, des notes de restitution d'ateliers clients, ainsi que les volumineux fichiers CSV servant de thésaurus pour l'application de l'ONaCVG.

Cependant, l'expérience utilisateur offerte par l'interface web de SharePoint contrastait fortement avec la vélocité de mon environnement de développement sous Linux. Naviguer au sein de ces dossiers imbriqués s'avérait particulièrement laborieux en comparaison de la fluidité offerte par des outils de navigation en ligne de commande basés sur des recherches floues, tels que *fzf* ou *zoxide*, que j'utilise quotidiennement dans mon terminal.

Par ailleurs, la capitalisation de la connaissance technique souffrait d'un certain morcellement. Si les règles métiers résidaient sur SharePoint, la documentation technique était éparpillée entre les dépôts Git (via les fichiers *README.md*), la plateforme Bookstack de l'entreprise, et un Wiki interne distinct. Cette fragmentation imposait une gymnastique mentale constante pour retrouver la bonne information au bon endroit.

1.4.3 Inspection web et débogage dynamique

Si le terminal et Neovim constituent mon espace de création logique, le navigateur web représente l'environnement d'exécution final, le véritable « compilateur visuel » de mon travail. Au sein d'Actimage, bien que certains collaborateurs aient opté pour Mozilla Firefox ou Opera, j'ai fait le choix de conserver Google Chrome. Cette décision était principalement motivée par une continuité d'usage avec mon environnement personnel, mais aussi pour sa rapidité d'exécution et la modernité de son moteur.

Ma configuration du navigateur est restée minimaliste. La seule extension véritablement indispensable à mon flux de travail était Bitwarden, configurée pour se synchroniser avec l'Actipass, l'instance auto-hébergée du gestionnaire de mots de passe d'Actimage, garantissant un accès sécurisé aux différents environnements de développement et de pré-production. Avec le recul, la création de profils Chrome distincts (un personnel et un professionnel) aurait été judicieuse. Par manque d'isolation de contexte, mes favoris personnels et professionnels se sont retrouvés entremêlés. Pour pallier l'absence de profils isolés par projet et éviter les conflits de sessions ou d'états, j'ai pris l'habitude de recourir massivement à la navigation privée ou aux rechargements forcés avec vidage du cache afin de simuler le comportement d'un nouvel utilisateur vierge de tout historique.

Dans l'écosystème Symfony, une grande partie du débogage est facilitée par l'excellent profileur Symfony, une barre d'outils injectée en bas de page offrant une visibilité totale sur le cycle de vie de la requête HTTP (requêtes SQL exécutées, temps de réponse, formulaires soumis, événements déclenchés). Néanmoins, le profileur atteint ses limites dès lors qu'il s'agit d'inspecter le comportement du code exécuté côté client. C'est ici que les outils de développement intégrés à Chrome (*DevTools*) prenaient le relais.

Je sollicitais les *DevTools* pour plusieurs cas d'usage bien précis :

L'onglet Éléments (Elements)

Il s'est avéré particulièrement redoutable pour le développement de l'interface utilisateur. Utilisant le cadriciel CSS utilitaire Tailwind, je pouvais manipuler les attributs de classe des éléments HTML à la volée directement

dans le DOM. Cette méthode permet de valider instantanément un comportement visuel ou un ajustement responsif dans le navigateur, avant d’aller inscrire la classe correspondante en dur dans les gabarits Twig.

L’onglet Sources

Je le consultais régulièrement pour m’assurer que le navigateur avait bien récupéré les dernières versions compilées de mes scripts JavaScript et feuilles de style, un point de vérification essentiel lors de l’utilisation du composant Symfony AssetMapper qui gère le versionnage des fichiers statiques.

L’onglet Réseau (Network)

Bien que le profileur Symfony permette d’analyser le temps des requêtes, l’onglet Réseau de Chrome était très pratique pour visualiser les requêtes asynchrones en cascade, notamment lors de l’ingestion de lourdes charges de données ou lors des appels API du formulaire d’inspection de l’ONaCVG.

La Console

Elle venait combler une lacune majeure du profileur Symfony : l’absence d’agrégation des erreurs JavaScript. La console était mon outil de diagnostic principal pour traquer les avertissements, lire les erreurs remontées par les contrôleurs Stimulus (Symfony UX), ou exécuter rapidement des requêtes exploratoires sur des objets du DOM.

Enfin, concernant le débogage côté serveur, l’outil Xdebug était bien configuré et présent dans la pile Docker de l’application ONaCVG. Toutefois, je n’en ai eu qu’un usage extrêmement marginal. La combinaison d’une analyse statique très stricte (PHPStan), garantissant la cohérence des types et de la logique structurelle en amont, couplée à la richesse d’informations délivrées par le profileur Symfony et aux tests unitaires, permettait d’identifier l’origine des anomalies sans avoir à recourir à l’exécution pas-à-pas offerte par un débogueur traditionnel.

I.4.4 L’éditeur de code: le choix de Neovim

Face à la complexité de l’écosystème PHP/Symfony, j’ai dans un premier temps évalué l’utilisation d’un environnement de développement intégré (IDE) classique, tel que PhpStorm [3]. Bien que cet outil propose des intégrations natives puissantes, sa lourdeur d’exécution et son manque de flexibilité (même pallié par l’extension IdeaVim [4]) ont constitué des freins à ma productivité. J’ai donc réintégré Neovim [5] comme éditeur principal.

Le langage PHP étant interprété et historiquement peu strict sur le typage, il ne bénéficie pas nativement des mêmes garanties à l’écriture qu’un langage compilé. Pour pallier cela et atteindre une rigueur de développement similaire à celle qu’offre TypeScript dans l’écosystème JavaScript, j’ai dû configurer un outillage robuste basé sur le Language Server Protocol (LSP) [6].

- J’ai principalement utilisé Phpactor [7] et Intelephense [8] pour l’autocomplétion et l’analyse statique.
- J’ai également expérimenté avec PHPantom [9] un serveur de langage récent développé en Rust [10], offrant des performances d’exécution nettement supérieures.
- L’intégration récente d’un serveur de langage dédié spécifiquement à Symfony [11] est venue parfaire cette configuration, me permettant d’avoir un retour intelligent sur les spécificités du cadriceil.

L’analyse de la qualité du code est restée isolée localement pour chaque projet, tout en étant exploitée par les serveurs de langage pour remonter les erreurs directement dans l’éditeur.

I.4.5 Contributions aux logiciels libres et outillage sur mesure

Constatant que certains outils de l’écosystème manquaient de maturité par rapport à d’autres langages, j’ai adopté une démarche proactive en contribuant à des projets à code source ouvert. J’ai notamment signalé et résolu des anomalies sur Twiggy [12] (un serveur de langage pour le moteur de gabarits Twig) et amélioré la grammaire Tree-sitter [13] de Twig, utilisée par Neovim pour l’analyse syntaxique. Cette implication m’a

également amené à échanger avec Fabien Potencier, créateur de Symfony, autour du développement de leur nouveau serveur de langage.

Pour optimiser la navigation dans les bases de code PHP, j'ai également développé des requêtes Tree-sitter personnalisées pour le greffon vim-matchup [14]. Celles-ci offrent une navigation syntaxique avancée en définissant des portées spécifiques pour les balises et les structures de contrôle du langage. Il devient ainsi possible de naviguer intelligemment entre les différentes clauses d'une condition ou d'une boucle, de parcourir aisément les blocs complexes (tels que switch, match ou try/catch), et de sauter instantanément de la signature d'une fonction à ses instructions de retour.

I.4.6 Le terminal comme espace de travail unifié

La reproduction de mon environnement s'arrêtant aux frontières du WSL et donc du terminal, j'ai optimisé ce dernier pour limiter au maximum l'usage de la souris et la friction liée aux changements de contexte. Le multiplexeur tmux [15] a été central dans cette démarche en me permettant de gérer de multiples invites de commande au sein d'une même fenêtre.

Afin de fluidifier mon flux de travail, j'ai enrichi cet espace de scripts et de raccourcis sur mesure. Ces personnalisations me permettent d'invoquer des interfaces interactives telles que lazygit [16] et lazydocker [17] sous forme de fenêtres superposées sans jamais quitter mon contexte d'édition, mais également d'interagir nativement au clavier avec des liens hypertextes ou d'accéder instantanément à l'interface web du dépôt Git du projet courant.

I.4.7 Conteneurisation et exécution locale

L'ensemble des projets, tels que PIAWEB, s'exécutaient au sein de conteneurs Docker [18]. Après une première phase d'utilisation de Docker Desktop pour Windows, j'ai rapidement constaté un manque de granularité et une interface graphique ralentissant mon flux de travail.

J'ai par conséquent migré vers une installation native du démon Docker exclusivement au sein de WSL. Ce choix m'a garanti un contrôle absolu sur les ressources et les conteneurs, pilotables intégralement en ligne de commande ou via l'interface terminale de lazydocker, en parfaite adéquation avec le reste de mon outillage.

I.4.8 Évolution du poste de travail

Travaillant pour la première fois dans un contexte extra-scolaire et extra-personnel pendant aussi longtemps, je trouve encore chaque semaine des points de friction, des tâches répétitives à optimiser ou automatiser. Mon poste de travail est en évolution constante, s'adaptant aux besoins de mon environnement de travail et de mes projets.

II Projet ONaCVG

Le projet principal sur lequel j'ai eu l'occasion de travailler tout au long des 6 mois de stage est une application web à destination de l'ONaCVG. **TODO: étoffer**

II.1 Introduction

II.1.1 Présentation du client et genèse du besoin

L'ONaCVG est un établissement public administratif français placé sous la tutelle du ministère des Armées et des Anciens Combattants [1]. Fondé en 1916, cet organisme a pour vocation d'assurer des missions de reconnaissance, de réparation, de solidarité et de mémoire envers les combattants, les anciens combattants et les victimes de guerre [1]. L'Office opère au bénéfice d'environ 1,81 million de ressortissants (selon des estimations de 2023) à travers un réseau de services de proximité et s'impose comme l'opérateur majeur de la politique mémorielle du ministère des Armées [1].

Parmi ses prérogatives, l'ONaCVG exerce une compétence juridique spécifique en matière de sépultures militaires, un domaine encadré par le Code des pensions militaires d'invalidité et des victimes de guerre (CPMIVG) [1]. L'institution est explicitement chargée de la mise en œuvre de l'entretien, de la rénovation et de la valorisation des sépultures de guerre [1]. En effet, la loi pose le principe d'une sépulture perpétuelle pour les militaires déclarés « Mort pour la France », qu'ils reposent au sein de nécropoles nationales ou de carrés militaires communaux, et dont l'entretien incombe à l'État [1].

C'est dans le cadre de la gestion de ce vaste patrimoine funéraire et historique, et afin de moderniser ses outils numériques, que l'institution a lancé un appel d'offres visant à concevoir une nouvelle application centralisée de gestion des sépultures. Ce marché a été remporté par Actimage en **TODO: insérer date**. Le périmètre du contrat couvre la conception, le développement, la tierce maintenance applicative (TMA) ainsi que l'hébergement du futur service. L'application logicielle développée est exclusivement destinée à un usage interne, ses utilisateurs finaux étant principalement les chefs de secteur de l'ONaCVG œuvrant sur le terrain et les administrateurs du pôle ECM.

II.1.2 L'existant : un défi de taille et de structure de la donnée

Le principal enjeu de ce projet réside dans l'héritage technique des données. La base de données existante (sous format MS Access [19]) recense plus de 800 000 sépultures, dont certaines remontent aux guerres napoléoniennes. Cette base historique compile une multitude d'informations : état civil militaire, nom et type du site, mentions honorifiques (telles que « Mort pour la France »), nationalité, informations de recrutement, unité militaire, ou encore causes du décès.

Cependant, le départ de la personne en charge de sa maintenance a entraîné une dégradation de l'intégrité des données, transformant la base en un document tabulaire peu rigoureux. Pour pallier ce manque d'outil centralisé, plusieurs chefs de secteur avaient dupliqué la « base mère » pour maintenir leurs données localement. Cette pratique a conduit à l'émergence de multiples « bases filles » désynchronisées, comportant des identifiants conflictuels, des doublons et des incohérences.

II.1.3 Migration et regroupement familial

La première mission de mon stage, qui s'est étendue sur un mois, a consisté à développer un outil de migration indépendant. Son objectif était de regrouper les bases filles avec la base mère en détectant les conflits et en proposant des stratégies de résolution : historisation des entrées conflictuelles ou conservation des deux versions via une renumérotation intelligente. Ce premier projet, qui fera l'objet d'une section détaillée ultérieurement, a constitué une excellente porte d'entrée pour m'approprier l'environnement technique de l'entreprise (PHP, Symfony, Doctrine, PostgreSQL, Docker) et les données de l'ONaCVG.

II.1.4 Refonte logicielle et application de gestion ECM.

Une fois les données fusionnées (toujours sous un format tabulaire plat d'environ quarante colonnes), la mission principale de mon stage a pu débuter : le développement de l'application de gestion complète, structurée autour de trois grands axes fonctionnels.

Modélisation et consultation (Base ECM)

Afin d'éviter la duplication et de garantir l'intégrité future des données, une refonte complète du modèle de données a été nécessaire. Nous sommes passés d'un format plat hérité du CSV à une architecture relationnelle stricte (création d'entités distinctes pour les pays, départements, communes, grades, unités, bureaux de recrutement, etc.). Une part majeure de mon travail a été consacrée à l'élaboration de la commande d'importation, capable de transformer des données libres et peu rigoureuses en entités standardisées. Sur cette base saine, un module de consultation a été développé, offrant des interfaces de recherche avancée avec de multiples filtres pour explorer les données des soldats et des sites.

Module d'inspection en mobilité

L'ONaCVG ayant la charge de sépultures à perpétuité, les chefs de secteur doivent inspecter leurs sites (parfois plus de 300 par secteur) au moins une fois par an. J'ai participé au développement d'une interface optimisée pour tablettes permettant la saisie d'inspections sur le terrain. L'agent peut y corriger les informations de la base et évaluer l'état des infrastructures (sol, barrières, stèles, plaques). Ces relevés alimentent ensuite un algorithme de calcul estimant les coûts de restauration pour les tombes et sites concernés.

Administration et flux de validation

Le troisième volet de l'application concerne les administrateurs ECM. Pour garantir la qualité de la base de données sur le long terme, un flux de travail (workflow) a été mis en place. Bien que certaines actions soient libres, la modification de champs sensibles par un chef de secteur nécessite l'approbation d'un administrateur. Ce processus est accompagné d'un système de notifications intra-application et de courriels automatisés.

II.1.5 Contraintes d'interopérabilité

Enfin, le système devait respecter une contrainte forte d'interopérabilité avec les services de l'État. Les données n'étant pas strictement confidentielles, elles sont rendues accessibles au grand public via le portail gouvernemental Mémoire des Hommes⁷. L'application développée intègre donc une fonctionnalité d'export mensuel générant un format de fichier très spécifique, garantissant l'alimentation continue et conforme de ce portail national.

Interlocuteurs, équipe et gestion de projet

Dans le cadre de ce projet, nos interlocuteurs de l'ONaCVG étaient Audrey Paolasini, cheffe du département des achats, Emmanuelle PORTUGAL, archiviste, **TODO: le reste.**

Dans la réalisation de ce projet, j'étais accompagné de Matthias en tant que chef de projet, Marine responsable de l'UI/UX, Aurélie en assistance chefferie de projet, rédaction de spécifications et également en développement **TODO: demander son rôle exact**, Brice en tant que *lead developer*, et Amine et moi-même en tant que développeurs.

Pour chaque fonctionnalité majeure de l'application ont lieu des ateliers entre nos interlocuteurs et Matthias, Marine et Brice. Ces ateliers précisent le cahier des charges initial de l'appel d'offre. En découlent des maquettes Figma [20] que Marine réalise et puis des spécifications fonctionnelles détaillées (SFD) basées sur les maquettes et le cahier des charges. Ensuite, Brice divise la fonctionnalité en tickets et les assigne à Amine ou moi en fonction de nos capacités et disponibilités.

⁷<https://memoiredeshommes.defense.gouv.fr>

II.2 Migration et regroupement familial : le défi de la réconciliation des données

Cette phase du projet a été particulièrement formatrice, marquant ma première immersion dans l'écosystème PHP, le cadriciel Symfony et l'ORM Doctrine. Le défi à relever consistait à consolider une « base mère » et de multiples « bases filles » (fournies sous forme de fichiers CSV). Au sein de ces bases, chaque sépulture est théoriquement identifiée par un entier unique : la colonne `sdr_num` (numéro de saisie des registres). Cependant, suite à la scission des bases et à l'ajout décentralisé de nouvelles entrées par les chefs de secteur, de nombreuses collisions d'identifiants sont apparues.

Une simple fusion automatisée était inenvisageable en raison de la nature ambiguë de ces collisions. Si deux lignes strictement identiques peuvent être dédoublonnées sans risque, le cas de lignes partageant le même `sdr_num` mais présentant des divergences s'avère complexe. Il peut s'agir d'une véritable collision (deux soldats distincts ayant reçu le même identifiant de manière isolée) ou d'une mise à jour légitime (un même soldat dont les informations ont été enrichies dans une base fille, par exemple avec l'ajout d'un surnom). L'absence de règle mathématique pour trancher ces cas a imposé le développement d'un outil de migration interactif. Cet outil agit comme une preuve de concept (POC) destinée à détecter les conflits et à déléguer la stratégie de résolution à l'utilisateur.

II.2.1 Choix technologiques et rigueur logicielle

S'agissant d'un projet de réconciliation de données critiques, il était primordial d'établir des fondations techniques solides. J'ai configuré ce projet sur les dernières normes de l'écosystème : PHP 8.2 couplé à Symfony 7.4 et PostgreSQL 18.

Afin de garantir la maintenabilité et la robustesse du code, j'ai intégré un outillage de qualité logicielle (QA) exigeant. L'analyse statique du code est assurée par PHPStan poussé à son niveau de vérification maximal (niveau 10), garantissant un typage strict et prévenant les erreurs d'exécution en amont. Le formatage du code est automatisé via PHP CS Fixer, et la fiabilité des algorithmes de résolution de conflits est couverte par des tests unitaires exécutés via PHPUnit. L'interface utilisateur, bien que secondaire pour un POC, utilise le moteur de gabarits Twig couplé au cadriciel Tailwind CSS via le composant Symfony AssetMapper, permettant de se passer d'une lourde chaîne de compilation JavaScript type Node.js/Webpack.

II.2.2 Architecture transactionnelle et asynchrone

L'ingestion de fichiers CSV contenant des centaines de milliers de lignes pose un défi architectural majeur : le traitement synchrone lors d'une requête HTTP entraîne inévitablement un dépassement du temps d'exécution autorisé (timeout) ou une saturation de la mémoire vive.

Pour y répondre, j'ai conçu une architecture asynchrone reposant sur le composant `symfony/messenger` et la bibliothèque d'extraction de données `league/csv`. Le flux de traitement s'articule autour de quatre tables PostgreSQL (TMP, MERE, CONFLICTS et HIST) et se déroule en plusieurs étapes :

Étape 1 : Acquisition et délégation

L'utilisateur téléverse un fichier CSV via la route `/csv-upload`. L'application sauvegarde le fichier sur le disque et délègue immédiatement le travail en publiant un message dans une file d'attente (gérée via le transport Doctrine), libérant ainsi le processus HTTP.

Étape 2 : Traitement asynchrone et optimisation mémoire

Un processus en tâche de fond (le « Worker ») consomme le message. Il utilise des itérateurs générateurs (via `league/csv`) pour lire le fichier ligne par ligne sans jamais charger l'intégralité des données en mémoire. Les lignes sont insérées par lots (batch) dans la table temporaire TMP.

Étape 3 : Répartition automatique

Un algorithme SQL compare ensuite les entrées de TMP avec celles de la base MERE. Les lignes sans conflit d'identifiant y sont intégrées directement. En revanche, les lignes soulevant une collision sur le `sdr_num` sont isolées dans la table CONFLICTS. La table TMP est ensuite purgée de l'entrée fille.

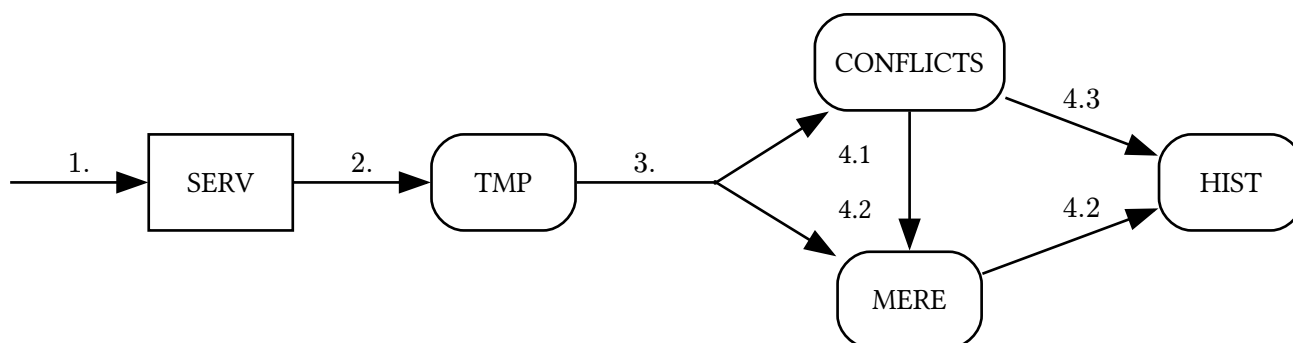


Fig. 2. – Flux de traitement de migration et résolution de conflits

II.2.3 Stratégies de résolution des conflits

L'application propose des interfaces web paginées pour lister et afficher les écarts en exergue (`routes / conflicts` et `/conflicts/{sdrNum}`). Face à une collision, l'utilisateur dispose de trois stratégies de résolution (opérées via l'API `/resolve/{origDb}/{sdrNum}`) :

Action 4.1 : Insertion

Si les deux entrées représentent des soldats différents (vraie collision), l'entrée fille est insérée dans la base MERE (génération d'un nouvel identifiant) et supprimée de CONFLICTS.

Action 4.2 : Écrasement

Si l'entrée fille est une version enrichie et valide de l'entrée mère, l'ancienne entrée mère est archivée dans la table d'historisation HIST. L'entrée fille vient ensuite la remplacer dans MERE via une opération de mise à jour (UPSERT), puis est supprimée de CONFLICTS.

Action 4.3 : Suppression

Si l'entrée fille est jugée erronée ou non pertinente, elle est retirée de la table CONFLICTS et sauvegardée dans HIST afin de conserver une trace de la donnée écartée en cas de besoin futur.

II.2.4 Ingénierie du déploiement et conteneurisation

Pour assurer l'isolation des processus et garantir la reproductibilité de l'environnement, l'ensemble de ce système a été pensé sous forme de micro-services via Docker Compose. L'environnement orchestre cinq conteneurs interdépendants :

- **db** : Le moteur de base de données PostgreSQL 18.
- **app** : Le conteneur principal exécutant l'application Symfony.
- **messenger-worker** : Un conteneur dédié exclusivement à la consommation des tâches asynchrones en arrière-plan (ingestion des CSV).
- **tailwind** : Un processus en mode « watch » recompilant à la volée les feuilles de style lors de la modification de l'interface.
- **nginx** : Le serveur web frontal agissant comme proxy inverse pour exposer l'application sur le port 8080.

II.2.5 Limites du modèle de données plat

Cette phase initiale d'ingestion a mis en évidence les limites physiques du format d'origine. Les exports CSV de la base mère, parfois volumineux (certains générant près de 80 000 conflits et représentant plus de 850 000 entrées au total), étaient traduits littéralement en colonnes PostgreSQL.

Tester l'outil sur de tels volumes a mis en exergue des problématiques de redondance de la donnée et des limites d'indexation. Cela a prouvé la nécessité absolue d'engager, pour le cœur de l'application ECM qui allait suivre, un travail profond de normalisation de la base de données (scission des champs libres en entités fortes avec clés primaires et jointures), sous peine de subir des temps de latence rédhibitoires lors des futures recherches et consultations.

II.3 Application principale : conception et réalisation

Une fois la consolidation des données historiques achevée, la phase majeure du projet a consisté à concevoir et développer l'application principale de suivi. Le périmètre fonctionnel, défini par de rigoureuses spécifications, englobe la consultation et la gestion de la base de l'État Civil Militaire (ECM), la saisie d'inspections sur le terrain via tablette, le pilotage budgétaire et la gestion des commandes de plaques.

Bien que l'implémentation de l'ensemble de ces fonctionnalités s'inscrive dans une feuille de route à long terme, mon travail s'est concentré sur la mise en place d'une architecture robuste, d'une chaîne d'intégration continue exigeante et sur le développement des modules centraux.

II.3.1 Architecture technique et paradigme « Front-End »

Pour répondre aux enjeux de maintenabilité et de pérennité du client, la pile technologique s'articule autour des dernières normes de l'écosystème PHP : Symfony 7.4 et PHP 8.5, adossés à une base de données PostgreSQL 18.

L'un des choix architecturaux majeurs a été l'abandon des chaînes de compilation JavaScript lourdes (de type Node.js/Webpack) au profit du composant Symfony AssetMapper. Ce paradigme moderne permet de gérer les dépendances front-end (JavaScript et CSS) directement via PHP. L'interface utilisateur est ainsi propulsée par le moteur de gabarits Twig, stylisée dynamiquement via le cadriciel Tailwind CSS (compilé nativement), et rendue interactive grâce à Stimulus (Symfony UX). Cette approche réduit drastiquement la complexité de l'infrastructure de déploiement tout en garantissant des performances optimales côté client.

II.3.2 Rigueur logicielle et Intégration Continue (CI/CD)

Afin de garantir que le code produit par l'équipe réponde aux standards de qualité de l'ingénierie logicielle, j'ai participé à la mise en place d'une chaîne d'intégration et de déploiement continu (pipeline Jenkins) particulièrement stricte. Chaque demande de fusion (Merge Request) doit valider des étapes bloquantes avant d'être intégrée à la branche principale :

Analyse statique et Typage strict

Le code est analysé par PHPStan, configuré à son niveau d'exigence maximal (Niveau 9). Une ligne de base (baseline) a été générée pour la dette technique existante, forçant ainsi tout nouveau code à être irréprochable.

Formatage et refactorisation

La syntaxe est uniformisée automatiquement par PHP CS Fixer et Twig CS Fixer. Le code JavaScript est audité par Biome, et l'outil Rector est intégré en mode vérification (dry-run) pour suggérer des refactorisations architecturales (Dead code, Code quality, Type declarations).

Outre l'audit des dépendances Composer, la CI intègre Gitleaks, un outil scannant l'historique des modifications (via des expressions régulières) pour empêcher la fuite de secrets ou de clés d'API dans le code source.

Tests isolés

Les tests fonctionnels et unitaires sont propulsés par PHPUnit. L'utilisation du paquet `dama/doctrine-test-bundle` permet d'exécuter les tests de base de données au sein de transactions automatiquement annulées (rollback), garantissant l'isolation des tests et des performances d'exécution accrues.

II.3.3 Implémentation des règles métiers et sécurité

Le cahier des charges impose une gestion fine des habilitations, réparties selon plusieurs rôles : Superadmin, Administrateur ECM, Administrateur Inspection, Chef de secteur et différents profils de consultants. Les droits d'accès ont été modélisés via le composant Security de Symfony (Voters et hiérarchie des rôles), garantissant un accès cloisonné en lecture et en écriture selon l'étendue géographique de l'agent (National vs Secteur).

Le cœur du système repose sur la Base ECM, exigeant des interfaces de recherche multicritères complexes (par individu ou par site). La modélisation a nécessité de relier les entités Soldat et Site à un riche système de thésaurus (grades, conflits, unités, causes de décès) gérable dynamiquement par les administrateurs.

II.3.4 Défis techniques des modules fonctionnels

Bien que l'application comporte de nombreux modules, certains ont représenté des défis techniques et ergonomiques particulièrement intéressants :

Gestion des inspections en mobilité

L'application prévoit un module d'inspection destiné à être utilisé par les chefs de secteur sur tablette, directement sur les sites. L'interface a dû être pensée pour le format tactile (boutons larges, listes déroulantes optimisées). D'un point de vue technique, ce module intègre la capture de photographies via l'appareil de la tablette et l'application d'actions par lots (ex: dupliquer l'état de dégradation d'une stèle sur une plage de tombes définie par leurs numéros de rangs et de carrés).

Les sites étant souvent situés dans des endroits reculés, en particulier pour les carrés militaires que l'on peut trouver dans des cimetières de villages, voire de hameaux, le chef de secteur les inspectant n'aura pas toujours accès à internet. La solution envisagée est le téléchargement préalable des données nécessaires dans le cache du navigateur, permettant ainsi un usage totalement hors-ligne. Une fois l'inspection terminée et la tablette placée dans un lieu avec accès internet, le formulaire d'inspection peut être soumis et enregistré sur la base de données de l'application.

Flux de validation (Workflow)

Pour protéger l'intégrité des données historiques (sépultures perpétuelles, mentions « Mort pour la France »), les chefs de secteur ne peuvent pas altérer directement les champs sensibles. Un flux de validation a été implémenté : la modification soumise génère une demande de révision (avec système de notifications et d'emails gérés via symfony/mailer) que l'Administrateur ECM doit valider ou refuser avec justification depuis un tableau de bord dédié.

Interopérabilité et Exports

L'application devant alimenter le portail national « Mémoire des Hommes », un système d'export sur mesure a été pensé. Ce système génère de manière asynchrone des fichiers formats plats encapsulant les dernières modifications réglementaires et l'état des sites.

III PIAWEB : Une histoire de DevOps

L'application PIAWEB⁸ dont Actimage réalise les développements et dirige les déploiements sur les serveurs clients est un projet de répertoire pour suivre les différentes actions du plan d'investissement France 2030 qui relèvent spécifiquement du Ministère de l'Enseignement Supérieur et de la Recherche (MESR).

Ce projet est actuellement en phase de TMA, peu de développements sont réalisés, majoritairement des corrections de bogues ou des évolutions mineures. La pile technologique repose sur du Spring et du Angular, une solution légèrement plus élaborée que celle proposée pour l'ONaCVG puisqu'elle requiert deux conte-neurs serveur distincts pour le web frontal et dorsal.

III.1 TODO: find title

III.1.1 Architecture de l'environnement DevOps et de l'infrastructure

L'hébergement et le déploiement de PIAWEB s'appuient sur une infrastructure robuste et standardisée, représentative des bonnes pratiques du pôle Digital d'Actimage. L'ensemble des environnements est conte-neurisé à l'aide de Docker, ce qui garantit une isolation parfaite des processus et une reproductibilité des déploiements.

L'architecture matérielle du serveur alloué au projet se distingue par la présence d'un disque supplémentaire de 100 Go, géré et partitionné dynamiquement via LVM (Logical Volume Manager). Cet espace est stratégi-quement découpé :

- Un volume LVM de 70 Go est monté sur le répertoire `/srv/`. Ce répertoire centralise l'installation de Docker, les différentes instances déployées du projet, ainsi que les exécuteurs (runners) GitLab. Par défaut, le répertoire d'installation de Docker a d'ailleurs été reconfiguré pour pointer vers `/srv/docker` afin d'exploiter cet espace de stockage étendu.
- Un second volume LVM de 30 Go est monté sur `/srv/registry/` et est exclusivement dédié aux besoins du registre d'images (Harbor), permettant de stocker les images Docker générées.

L'accès aux différents services web est orchestré par un serveur mandataire inverse (reverse-proxy) Nginx. Les configurations d'accès sont gérées classiquement via les répertoires `/etc/nginx/sites-available` et `sites-enabled`. Afin de protéger les environnements hors production, une restriction d'accès de type `auth_basic` est implémentée. Différents fichiers d'identifiants (`.htpasswd_actimage`, `.htpasswd_client`, `.htpasswd_all`) sont utilisés pour cloisonner l'accès selon qu'il s'agisse de l'environnement d'intégration (réservé à Actimage) ou de recette (ouvert au client).

La pile applicative elle-même est divisée en trois conteneurs principaux :

Backend

Développé en Java 17 avec Springboot 2.7, ce module expose les services de l'application et se connecte à la base de données.

Frontend

Les interfaces utilisateur en Angular, dont les composants transpilés sont servis par un serveur Nginx interne et autonome.

Flyway

Un conteneur éphémère dédié exclusivement à la gestion et à l'exécution des scripts de migration SQL pour faire évoluer la structure de la base de données PostgreSQL à chaque changement des entités qui cause un changement de schéma de table. Ce conteneur se lance lorsque l'application est mise en ligne et s'arrête dès que les migrations sont effectuées.

⁸Contraction de Programme d'Investissements d'Avenir (PIA) et de Web.

TODO: Insérer ici un schéma DOT représentant l'architecture physique et logique de l'infrastructure PIAWEB.

Exemple de contenu DOT :

```
digraph architecture {
    rankdir=LR;
    Internet -> Nginx [label='HTTPS'];
    Nginx -> 'Frontend (Angular)' [label='Proxy'];
    Nginx -> 'Backend (Spring)' [label='API'];
    'Backend (Spring)' -> 'PostgreSQL' [label='JDBC'];
    'Flyway' -> 'PostgreSQL' [label='Migration'];
}
```

III.1.2 Intégration Continue et Déploiement Continu (CI/CD)

Afin de fluidifier le cycle de développement et de garantir la qualité du code, le projet s'appuie sur la plateforme GitLab pour son intégration continue. L'organisation du code est modulaire : un macro-projet centralise la configuration CI/CD, tandis que les différents composants (frontend, backend, base de données, infrastructure Docker) sont gérés sous forme de sous-modules Git.

L'exécution des tâches de la CI/CD est confiée à des GitLab Runners installés manuellement sur le serveur d'intégration. L'environnement s'appuie sur deux types d'exécuteurs :

- Un **runner Docker** (runner-gitlab-piaweb-int) : utilisé pour exécuter les tâches dans des conteneurs isolés, garantissant des environnements de construction propres et jetables.
- Un **runner Shell** (runner-gitlab-piaweb-shell) : configuré pour s'exécuter directement sur la machine hôte via un utilisateur système dédié (gitlab-runner). Bien que son usage soit généralement déconseillé car il peut laisser des fichiers résiduels, il est parfois nécessaire pour interagir directement avec l'infrastructure du serveur d'intégration.

Les pipelines (ou chaînes de traitement) sont configurés pour se déclencher selon des événements précis : lors de la soumission de code sur une branche spécifique, lors de la création d'une étiquette (tag), ou lors d'une publication (release).

III.1.3 Gestion des environnements et stratégie de branche

Le cycle de vie du code de PIAWEB est rythmé par le passage à travers différents environnements, chacun répondant à un besoin spécifique et associé à des stratégies de branches Git rigoureuses :

De dev vers int (Intégration)

Les environnements de développement local et d'intégration sont techniquement identiques. Ils exploitent les outils de rechargement à chaud (comme Springboot DevTool). À ce stade, le code source n'est pas « figé » dans les images Docker, mais monté via des volumes, ce qui évite de devoir reconstruire les images à chaque modification et accélère considérablement le cycle de développement. La mise à jour de l'environnement d'intégration se fait via de simples commandes `git pull` et `git submodule update`, suivies d'une compilation Maven (`mvn clean install`), tout ceci facilité par l'usage de clés de déploiement (Deploy Keys) configurées sur la machine virtuelle.

De int vers rec (Recette)

C'est lors du passage en recette que le paradigme change radicalement. Le code applicatif est désormais compilé et intégré en dur au sein même des images Docker. Ce figeage garantit que l'image testée par le client sera strictement identique à celle qui sera mise en production.

De rec vers prep (Pré-production)

Il s'agit d'environnements différents, mais qui exploitent les mêmes images Docker. Cette étape permet de valider le comportement de la version packagée dans une infrastructure imitant la production.

L'environnement et les images Docker restent les mêmes que lors de l'étape de pré-production. L'enjeu ici n'est plus technique mais critique : appliquer la mise à jour sans provoquer d'interruption de service ou d'anomalie sur le système en exploitation.

TODO: schéma des différentes branches Git

III.1.4 Le processus de livraison et de montée de version

La livraison d'une nouvelle version de PIWEB est une procédure méticuleuse qui demande rigueur et précision. La responsabilité de la création des livrables incombe à l'utilisateur système piaweb directement sur le serveur.

La procédure débute par la récupération des dernières modifications du code source depuis la branche de recette (rec) et de tous ses sous-modules. Le code dorsal est ensuite compilé via l'outil Maven (`mvn clean install`). Une fois les binaires générés, la construction des nouvelles images Docker (backend, frontend, et flyway) est lancée en désactivant le cache pour forcer une reconstruction totale.

Ces nouvelles images sont ensuite étiquetées (taggées) avec le numéro de version correspondant (par exemple `$CI_COMMIT_TAG`) et envoyées (poussées) vers le registre d'images privé d'Actimage (piaweb-registry.actimage-ext.net). L'accès à ce registre est protégé et nécessite une authentification préalable via la commande `docker login`.

Une fois les livrables créés et sécurisés sur le registre, la mise à jour effective de l'environnement (la montée de version) peut avoir lieu. Ce processus suit une chorégraphie immuable :

Arrêt des services

Les modules applicatifs en cours d'exécution sont stoppés proprement grâce à des scripts dédiés (`stop.sh`) présents dans l'arborescence de chaque composant.

Sauvegarde

Avant toute manipulation des données, une sauvegarde complète de la base de données PostgreSQL est réalisée (via un export `pg_dump` compressé en `.zst`), garantissant un point de restauration en cas de problème.

Configuration

Les fichiers de configuration d'environnement (`host.env`) des différents modules sont édités pour faire pointer la variable `VERSION` vers le nouvel identifiant de l'image Docker fraîchement produite.

Migration des données

Le script de migration Flyway (`flyway.sh migrate`) est exécuté. Il se charge d'appliquer séquentiellement les nouveaux scripts SQL nécessaires à la mise à jour de la structure ou des données de la base, tout en traçant son exécution.

Démarrage et contrôle

Les modules backend et frontend sont finalement relancés via leurs scripts `start.sh`. Le bon déroulement de l'opération est vérifié en inspectant les journaux de bord (logs) des conteneurs en temps réel. Il est impératif de lancer le module dorsal en premier puisqu'il peut fonctionner seul. Le module frontal, quant à lui, essaie automatiquement de se connecter au dorsal, pouvant entraîner un crash au démarrage si celui-ci n'est pas encore prêt à traiter des requêtes entrantes.

Si la recette est validée par le client, la version est promue en production. Les images testées sont simplement re-tagguées avec le préfixe `prod-` puis propulsées sur l'environnement de production, assurant ainsi qu'aucune modification de code n'a pu altérer l'application entre la phase de test et la mise en ligne finale.

TODO: Insérer ici un schéma de flux (flowchart) illustrant les étapes de la livraison :

**Code -> Build Maven -> Build Docker -> Docker Push (Registry) -> Stop Containers -> Backup DB
-> Config update -> Flyway Migrate -> Start Containers**

III.2 TODO: find title

III.2.1 Le bogue

Étant l'un des développeurs les moins coûteux, j'ai été assigné à la correction d'un bogue d'affichage ordinaire pour lequel plusieurs de mes collègues avaient déjà imputé du temps. La plongée laborieuse dans le code source d'un projet dont la teneur m'échappait encore et dont le cadrage frontal ne m'était pas familier m'a contraint à optimiser mon débogage afin d'identifier la source du problème sans avoir à explorer l'entièreté de l'application. Quelques échanges de tickets avec le client plus tard et j'arrivais à reproduire le comportement anormal sur mon poste. Le problème venait d'un simple formulaire de recherche dont la pagination n'était pas rapportée à 1 lorsque l'utilisateur changeait les critères de recherche, permettant ainsi d'accéder à la troisième page pour une recherche ne remontant qu'une page de résultats par exemple.

Si l'implémentation du correctif ne nécessita que quelques minutes, la validation de la demande de fusion sur la branche de développement dev fut actée en une demi-heure. L'intervention aurait pu s'achever sur cette bonne note, mais l'équipe DevOps m'a confié la responsabilité de l'intégralité du cycle de livraison de cette version, incluant la montée sur les différents environnements et le déploiement final chez le client.

III.2.2 Appareillage sur lest

Le cycle de livraison d'une version du site PIAWEB passe par plusieurs phases différentes, évoluant lentement de l'environnement de développement vers l'environnement de production.

IV Conclusion

IV.1 À propos de PHP/Symfony TODO: définir un titre

TODO: raconter ma life sur le typage, Twig etc, comparer à Java, Spring Boot et JS/TS

IV.2 Enrichissement personnel

TODO: parler de l'envie de bosser à la dinum, souveraineté numérique, FabPot

Bibliographie

- [1] Contributeurs de Wikipédia, « Office national des combattants et des victimes de guerre ». [En ligne]. Disponible sur: https://fr.wikipedia.org/wiki/Office_national_des_combattants_et_des_victimes_de_guerre

Références logicielles

- [2] Microsoft, « Windows Subsystem for Linux ». [En ligne]. Disponible sur: <https://wsl.dev/>
- [3] JetBrains, « PhpStorm ». [En ligne]. Disponible sur: <https://www.jetbrains.com/phpstorm>
- [4] JetBrains, « IdeaVim ». [En ligne]. Disponible sur: <https://lp.jetbrains.com/ideavim>
- [5] Contributeurs de Neovim, « Neovim ». [En ligne]. Disponible sur: <https://neovim.io/>
- [6] Microsoft, « LSP ». [En ligne]. Disponible sur: <https://microsoft.github.io/language-server-protocol>
- [7] Daniel Leech, « Phpactor ». [En ligne]. Disponible sur: <https://phpactor.readthedocs.io/>
- [8] Ben Mewburn, « Intelephense ». [En ligne]. Disponible sur: <https://intelephense.com/>
- [9] Anders Jenbo, « Phpantom ». [En ligne]. Disponible sur: https://phpantom-dev.github.io/phpantom_lsp
- [10] Rust Team, « Rust ». [En ligne]. Disponible sur: <https://rust-lang.org/>
- [11] Fabien Potencier, « Symfony Language Tools ». [En ligne]. Disponible sur: <https://github.com/symfony/language-tools>
- [12] Mikhail Gunin, « Twiggy ». [En ligne]. Disponible sur: <https://github.com/moetelo/twiggy>
- [13] Contributeurs de Tree-sitter, « Tree-sitter ». [En ligne]. Disponible sur: <https://tree-sitter.github.io/>
- [14] Andy Massimino, « Vim Matchup ». [En ligne]. Disponible sur: <https://github.com/andymass/vim-matchup>
- [15] Contributeurs de tmux, « tmux ». [En ligne]. Disponible sur: <https://github.com/tmux/tmux>
- [16] Jesse Duffield, « Lazygit ». [En ligne]. Disponible sur: <https://github.com/jesseduffield/lazygit>
- [17] Jesse Duffield, « Lazydocker ». [En ligne]. Disponible sur: <https://github.com/jesseduffield/lazydocker>
- [18] Docker Inc., « Docker ». [En ligne]. Disponible sur: <https://www.docker.com/>
- [19] Microsoft, « Microsoft Access ». [En ligne]. Disponible sur: <https://www.microsoft.com/fr-fr/microsoft-365/access>
- [20] Figma, « Figma ». [En ligne]. Disponible sur: <http://figma.com/>

V Annexes